

LAPORAN PRAKTIKUM PEKAN 1
PEMOGRAMAN BERORIENTASI OBJEK

*“Penerapan Class, Object, Constructor, Attribute
dan Method pada Sistem Perbankan Mini”*

disusun Oleh:

Nabil Fikri (2511533011)

Dosen Pengampu:

Nurfiah, S.ST, M.Kom.

Asisten Praktikum:

M. Dawi Syauqi



DEPARTEMEN INFORMATIKA
FAKULTAS TEKNOLOGI INFORMASI
UNIVERSITAS ANDALAS

2026

KATA PENGANTAR

Puji syukur penulis panjatkan ke hadirat Tuhan Yang Maha Esa, karena atas rahmat dan karunia-Nya penulis dapat menyelesaikan Laporan Praktikum Pekan 1 mata kuliah Pemrograman Berorientasi Objek dengan judul "Sistem Perbankan Mini" ini tepat pada waktunya.

Laporan ini disusun sebagai salah satu bentuk pertanggungjawaban tertulis atas kegiatan praktikum yang telah dilaksanakan, sekaligus sebagai sarana untuk mendokumentasikan pemahaman penulis mengenai konsep dasar pemrograman berorientasi objek, khususnya penerapan class, object, atribut, method, dan constructor dalam bahasa pemrograman Java.

Penulis menyadari bahwa dalam penyusunan laporan ini masih terdapat kekurangan, baik dari segi penulisan maupun pembahasan materi. Oleh karena itu, penulis dengan terbuka menerima kritik dan saran yang membangun guna perbaikan laporan pada kesempatan berikutnya.

Penulis juga mengucapkan terima kasih kepada dosen pengampu mata kuliah Pemrograman Berorientasi Objek yang telah memberikan bimbingan dan arahan selama proses praktikum berlangsung, sehingga laporan ini dapat terselesaikan dengan baik.

Akhir kata, penulis berharap laporan ini dapat bermanfaat bagi pembaca, khususnya dalam memahami penerapan konsep OOP melalui studi kasus program perbankan sederhana ini.

Padang, 14 September 2026

Nabil Fikri

BAB I

PENDAHULUAN

1.1 Latar Belakang

Praktikum Pekan 1 ini bertujuan untuk memahami konsep dasar pemrograman berorientasi objek (Object Oriented Programming/OOP), khususnya konsep **class**, **object**, **atribut**, **method**, dan **constructor** dalam bahasa pemrograman Java. Studi kasus yang digunakan adalah pembuatan program "Sistem Perbankan Mini" yang mensimulasikan proses pembukaan rekening, setor tunai, tarik tunai, dan pengecekan informasi rekening nasabah.

Program ini terdiri atas dua buah class, yaitu:

1. **Class Rekening**, yang berperan sebagai *blueprint* atau cetak biru objek rekening nasabah, berisi atribut dan method yang menggambarkan data serta perilaku dari sebuah rekening bank.
2. **Class Main**, yang berperan sebagai class utama (driver) untuk menjalankan program, menampilkan menu interaktif kepada pengguna, serta melakukan instansiasi dan pemanggilan method dari objek Rekening.

Melalui praktikum ini, mahasiswa diharapkan mampu memahami bagaimana sebuah objek dibuat dari class, bagaimana constructor bekerja saat objek diinisialisasi, serta bagaimana method digunakan untuk memanipulasi data (atribut) yang dimiliki oleh objek tersebut.

1.2 Tujuan Praktikum

1. Memahami konsep dasar pemrograman berorientasi objek (OOP), khususnya class, object, atribut, dan method dalam bahasa pemrograman Java.

2. Mampu membuat class yang merepresentasikan entitas dunia nyata (dalam hal ini rekening bank) beserta atribut dan perilakunya.
3. Memahami cara mengakses dan memanipulasi atribut suatu objek melalui pemanggilan method.
4. Mampu menerapkan struktur kontrol (percabangan dan perulangan) untuk membangun program interaktif berbasis menu.

1.3 Manfaat Praktikum

1. Mahasiswa memperoleh pengalaman praktis dalam merancang dan mengimplementasikan class serta objek dalam bahasa Java.
2. Mahasiswa lebih memahami relasi antara class model data (Rekening) dan class pengendali program (Main) dalam sebuah aplikasi.
3. Mahasiswa mampu mengaplikasikan konsep OOP dalam studi kasus nyata, seperti sistem perbankan sederhana, sehingga lebih memahami relevansi materi dengan penerapan di dunia kerja.
4. Mahasiswa terlatih dalam menerapkan logika pemrograman untuk melakukan validasi transaksi, seperti pengecekan saldo dan nominal minimum.

BAB II

PEMBAHASAN

2.1 Pengertian Class, Object, Contructor, Attribute, dan Method

2.1.1 Class

Class adalah cetak biru atau *blueprint* dari object. Class digunakan hanya untuk membuat kerangka dasar. Yang akan kita pakai nanti adalah hasil cetakan dari class, yakni object.

Sebagai analogi, class bisa diibaratkan dengan laptop atau notebook, yakni gambaran umum tentang sebuah benda. Di dalam pemrograman nanti, contoh class seperti User, Item, Siswa, Validate, dll.

Dalam bahasa Java, penulisan class diawali dengan *keyword* class, kemudian diikuti dengan nama dari class tersebut. Aturan penulisan nama class sama seperti aturan penulisan *variabel* di Java (lebih tepatnya aturan identifier), yakni tidak boleh diawali angka dan tidak boleh mengandung spasi.

2.1.2 Object

Object atau objek adalah hasil cetak dari class, atau bisa juga disebut hasil konkrit dari class. Masih menggunakan analogi class Laptop, maka object dari class Laptop bisa berupa: laptopNabil, laptopAtar, atau laptopSofian.

Sebuah object dari class Laptop akan memiliki seluruh ciri-ciri laptop, termasuk property dan method-nya. Proses mencetak object dari class ini disebut dengan instansiasi (atau instantiation).

Dalam bahasa Java, proses instansiasi dilakukan dengan menulis nama class, lalu diikuti nama variabel yang akan menampung object tersebut, tanda sama dengan "=", keyword new dan kembali nama class dengan tambahan tanda kurung "()".

Hasil cetakan class akan disimpan dalam variabel untuk selanjutnya dipakai selama penulisan kode program. Variabel inilah yang akan berisi object dari sebuah class.

2.1.3 Attribute/Property

Attribute (atau kadang juga dengan *Property* atau *field*) adalah data yang terdapat dalam sebuah class. Melanjutkan analogi tentang *laptop*, property dari laptop bisa berupa *merk*, *warna*, *jenis processor*, *ukuran layar*, dan lain-lain.

Jika anda sudah terbiasa dengan program Java, property ini sebenarnya hanyalah variabel yang terletak di dalam class. Seluruh aturan dan tipe data yang biasa diinput ke dalam variabel, juga bisa diinput ke dalam property. Aturan tata cara penamaan property sama dengan aturan penamaan variabel.

2.1.4 Method

Method adalah tindakan yang bisa dilakukan di dalam class. Jika menggunakan analogi class Laptop, maka contoh method adalah: *menghidupkan laptop*, *mematikan laptop*, atau *mengganti cover laptop*.

Method pada dasarnya adalah *function* yang berada di dalam class. Seluruh sifat function bisa diterapkan ke dalam method seperti bisa di isi argument/parameter, mengembalikan nilai (dengan keyword *return*), dan lain-lain.

2.1.5 Constructor

Constructor adalah sebuah method khusus di dalam sebuah class yang berfungsi untuk menginisialisasi (memberi nilai awal) pada attribute suatu objek saat objek tersebut pertama kali dibuat atau diinstansiasi menggunakan kata kunci *new*. Constructor akan dieksekusi secara otomatis satu kali saja, yaitu tepat pada saat proses pembuatan objek terjadi.

Ciri-ciri constructor:

1. Nama constructor harus sama persis dengan nama class tempatnya berada.
2. Tidak memiliki tipe data kembalian (return type) sama sekali, bahkan tidak boleh dituliskan void.
3. Dipanggil secara otomatis ketika objek dibuat dengan kata kunci new, tanpa perlu dipanggil secara manual seperti method biasa.
4. Berfungsi untuk mengisi nilai awal attribute agar objek yang baru dibuat langsung memiliki data yang valid dan siap digunakan.

2.2 Implementasi pada Praktikum

2.2.1 Struktur Class Rekening

```

1 package Pekan1;
2
3 public class Rekening {
4
5     String nomorRekening;
6     String namaPemilik;
7     double saldo;
8
9     public Rekening(String nomor, String nama, double saldoAwal) {
10         nomorRekening = nomor;
11         namaPemilik = nama;
12         saldo = saldoAwal;
13         System.out.println("Rekening atas nama " + namaPemilik + " berhasil dibuat dengan saldo Rp" + saldo);
14     }
15
16     public void setorTunai(double nominal) {
17         if (nominal > 0) {
18             saldo += nominal;
19             System.out.println("Setor tunai Rp" + nominal + " berhasil. Saldo saat ini : Rp" + saldo);
20         } else {
21             System.out.println("Gagal : Nominal setor harus lebih dari 0!");
22         }
23     }
24
25     public void tarikTunai(double nominal) {
26         if (nominal >= 10000) {
27             if (nominal > saldo) {
28                 System.out.println("Transaksi Gagal: Saldo tidak mencukupi. Saldo saat ini : Rp" + saldo);
29             } else {
30                 saldo -= nominal;
31                 System.out.println("Tarik tunai Rp" + nominal + " berhasil. Saldo saat ini : Rp" + saldo);
32             }
33         } else {
34             System.out.println("Transaksi Gagal : Minimal nominal penarikan Rp10.000");
35         }
36     }
37
38     public void cekInformasi() {
39         System.out.println("--- INFO REKENING ---");
40         System.out.println("No. Rekening : " + nomorRekening);
41         System.out.println("Nama Pemilik : " + namaPemilik);
42         System.out.println("Saldo Akhir : Rp" + saldo);
43         System.out.println("-----");
44     }
45 }

```

Gambar 2.1.1

Pada Class Rekening memiliki tiga atribut, yaitu nomorRekening, namaPemilik, dan saldo, yang masing-masing merepresentasikan data dari sebuah rekening bank. Class ini juga memiliki sebuah constructor Rekening(String nomor, String nama, double saldoAwal) yang akan otomatis dijalankan saat objek baru dibuat menggunakan kata kunci new. Constructor ini bertugas menginisialisasi nilai awal

dari ketiga atribut tersebut sekaligus menampilkan pesan konfirmasi bahwa rekening berhasil dibuat.

Class Rekening juga memiliki tiga method utama:

- **setorTunai(double nominal)** : menambahkan nilai saldo apabila nominal yang dimasukkan lebih besar dari nol; jika tidak, transaksi ditolak.
- **tarikTunai(double nominal)** : mengurangi nilai saldo dengan mempertimbangkan dua validasi, yaitu nominal penarikan minimal Rp10.000 dan saldo harus mencukupi sebelum transaksi diproses.
- **cekInformasi()** : menampilkan seluruh data rekening (nomor rekening, nama pemilik, dan saldo akhir) ke layar.

2.2.2 Struktur Class Main

```

1 package Pekan1;
2 import java.util.Scanner;
3
4
5 public class Main {
6     public static void main(String[] args) {
7         Scanner input = new Scanner(System.in);
8         Rekening akunAktif = null; // Objek belum diinisialisasi (null)
9         boolean isRunning = true;
10
11         System.out.println("=== SISTEM PERBANKAN MINI ===");
12
13         while (isRunning) {
14             System.out.println("\nMenu Utama : ");
15             System.out.println("1. Buka Rekening Baru");
16             System.out.println("2. Setor Tunai");
17             System.out.println("3. Tarik Tunai");
18             System.out.println("4. Cek Informasi Rekening");
19             System.out.println("0. Keluar");
20             System.out.print("Pilih Menu : ");
21
22             int pilihan = input.nextInt();
23             input.nextLine();
24
25             switch (pilihan) {
26                 case 1 :
27                     System.out.print("Masukkan No Rekening : ");
28                     String no = input.nextLine();
29                     System.out.print("Masukkan Nama Pemilik : ");
30                     String nama = input.nextLine();
31                     System.out.print("Masukkan Saldo Awal : ");
32                     double saldo = input.nextDouble();
33
34                     // Instansiasi Object / Mendalankan Konstruktor
35                     akunAktif = new Rekening(no, nama, saldo);
36                     break;
37
38                 case 2 :
39                     if (akunAktif == null) {
40                         System.out.println("Error: Mohon maaf, anda belum memiliki nomor rekening!");
41                     } else {
42                         System.out.print("Masukkan nominal setor: ");
43                         double setor = input.nextDouble();
44                         akunAktif.setorTunai(setor); // Mendalankan Behavior / method
45                     }
46                     break;
47
48                 case 3 :
49                     if (akunAktif == null) {
50                         System.out.println("Error: Mohon maaf, anda belum memiliki nomor rekening!");
51                     } else {
52                         System.out.print("Masukkan nominal tarik: ");
53                         double tarik = input.nextDouble();
54                         akunAktif.tarikTunai(tarik);
55                     }
56                     break;
57
58                 case 4 :
59                     if (akunAktif == null) {
60                         System.out.println("Error: Anda belum membuka rekening!");
61                     } else {
62                         akunAktif.cekInformasi();
63                     }
64                     break;
65
66                 case 0 :
67                     isRunning = false;
68                     System.out.println("Sistem ditutup. Terima Kasih!");
69                     break;
70
71                 default:
72                     System.out.println("Pilihan tidak valid!");
73             }
74         }
75         input.close();

```

Gambar 2.2.2

Pada Class Main(Driver) merupakan titik masuk (entry point) program yang menjalankan sebuah perulangan while untuk menampilkan menu utama secara berulang selama program belum dihentikan. Objek akunAktif bertipe Rekening pada awalnya diberi nilai null, yang menandakan belum ada rekening yang aktif atau terbentuk.

Program menyediakan lima pilihan menu melalui struktur switch-case:

- Menu 1 (Buka Rekening Baru) : meminta input nomor rekening, nama, dan saldo awal dari pengguna, kemudian melakukan instansiasi objek baru dari class Rekening yang otomatis memanggil constructor.
- Menu 2 (Setor Tunai) dan Menu 3 (Tarik Tunai) : memanggil method setorTunai() dan tarikTunai() dari objek akunAktif, namun terlebih dahulu memvalidasi apakah objek tersebut sudah diinisialisasi (tidak null).
- Menu 4 (Cek Informasi Rekening) : memanggil method cekInformasi() untuk menampilkan data rekening yang sedang aktif.
- Menu 0 (Keluar) : mengubah nilai variabel isRunning menjadi false sehingga perulangan while berhenti dan program selesai dijalankan.

2.2.3 Analisis Konsep

Dari kedua class tersebut dapat diidentifikasi beberapa penerapan konsep OOP, di antaranya:

- Encapsulation : data (atribut) dan perilaku (method) dikelompokkan dalam satu kesatuan class Rekening.
- Abstraction : pengguna program (melalui class Main) hanya berinteraksi dengan method-method publik tanpa perlu mengetahui detail implementasi di dalamnya.
- Constructor & Instansiasi Objek : objek akunAktif dibentuk dari class Rekening melalui pemanggilan constructor menggunakan kata kunci new.

- Validasi Data : setiap method transaksi (setor dan tarik tunai) dilengkapi dengan pengecekan kondisi (if-else) guna menjaga konsistensi dan keamanan data saldo.

BAB III

PENUTUP

3.1 Kesimpulan

Berdasarkan hasil praktikum yang telah dilakukan, dapat disimpulkan bahwa program "Sistem Perbankan Mini" berhasil mengimplementasikan konsep dasar pemrograman berorientasi objek dengan baik, khususnya dalam hal pemisahan tanggung jawab antara class model data (Rekening) dan class pengendali program (Main). Penerapan constructor mempermudah proses inisialisasi objek, sementara method-method yang disediakan memungkinkan manipulasi data saldo secara aman melalui berbagai validasi logika.

Melalui praktikum ini, mahasiswa memperoleh pemahaman praktis mengenai bagaimana sebuah objek diciptakan, digunakan, dan diinteraksikan dalam program Java, sekaligus memahami pentingnya validasi data dalam membangun sistem yang andal. Praktikum ini menjadi dasar penting untuk mempelajari konsep OOP lanjutan seperti inheritance, polymorphism, dan interface pada pertemuan berikutnya.

DAFTAR PUSTAKA

- [1] Andre, "Tutorial OOP Java Part 2: Pengertian Class, Object, Property dan Method," Duniaikom, 10 Oktober 2021. [Online]. Available: <https://www.duniaikom.com/tutorial-oop-java-pengertian-class-object-property-dan-method/>. [Accessed 15 September 2026].

LAPORAN TUGAS PRAKTIKUM

Pemograman Berorientasi Objek

Pekan 1 (*Class, Object, Constructor dan Method*)

Dosen Pengampu:

Nurfiah, S.ST, M.Kom.

Asisten Praktikum:

M. Dawi Syauqi



DEPARTEMEN INFORMATIKA

FAKULTAS TEKNOLOGI INFORMASI

UNIVERSITAS ANDALAS

2026

Deskripsi Tugas

1. Implementasi Fitur Tarik Tunai

- Tambahkan method tarikTunai(double nominal) ke dalam class Rekening.
- Aturan: Saldo tidak boleh menjadi minus dan minimal nominal Tarik tunai adalah 10.000. Jika pengguna mencoba menarik nominal yang lebih besar dari saldo saat ini atau nominal kecil dari 10.000, program tidak boleh crash/error, melainkan menampilkan peringatan ke konsol: "Transaksi Gagal: Saldo tidak mencukupi. Saldo Anda: Rp[nominal]" atau "Transaksi Gagal: Minimal nominal penarikan 10.000".
- Terapkan method ini dengan menu nomor 3 pada class Main.

2. Tantangan Multi-Akun Saat ini, aplikasi Main hanya dapat menyimpan 1 akun (Rekening akunAktif = null;). Jika pengguna memilih menu "Buka Rekening Baru" untuk kedua kalinya, akun lama akan tertimpa.

- Ubah arsitektur penyimpanan di Main.java menggunakan struktur data Collection. (Gunakan ArrayList).
- Ubah logika menu "Buka Rekening": Setiap kali pengguna membuka rekening baru, objek tersebut ditambahkan ke dalam ArrayList.
- (Bonus): Tambahkan menu baru untuk "Ganti Akun", di mana sistem akan meminta nomor rekening, kemudian mencari objek yang cocok di dalam ArrayList untuk dijadikan akunAktif

PEMBAHASAN TUGAS

1. Implementasi Fitur Tarik Tunai

Implementasi fitur Tarik tunai dilakukan pada class rekening, method dibuat hampir sama dengan method setor tunai, hanya saja berbeda pada pengurangan dan penambahan saldo. Namun, pada method tarikTunai juga harus memenuhi beberapa syarat yaitu minimal nominal penarikan adalah 10.000 dan tidak boleh melebihi dari saldo yang ada. Berikut adalah kode program method yang sesuai dan berdasarkan syarat tugas.

```
24 public void tarikTunai(double nominal) {  
25     if (nominal >= 10000) {  
26         if (nominal > saldo) {  
27             System.out.println("Transaksi Gagal: Saldo tidak mencukupi. Saldo saat ini : Rp" + saldo );  
28         } else {  
29             saldo -= nominal;  
30             System.out.println("Tarik tunai Rp" + nominal + " berhasil. Saldo saat ini : Rp" + saldo);  
31         }  
32     } else {  
33         System.out.println("Transaksi Gagal : Minimal nominal penarikan Rp10.000");  
34     }  
35 }
```

Fungsi tarikTunai menerima satu masukan berupa angka desimal bernama nominal, yang merepresentasikan jumlah uang yang ingin diambil oleh pengguna.

Ketika fungsi ini dijalankan, langkah pertama yang dilakukan adalah mengecek apakah nominal yang diminta memenuhi batas minimum penarikan, yaitu Rp10.000. Jika nominal ternyata kurang dari batas tersebut, sistem langsung membatalkan transaksi dan menampilkan pesan bahwa penarikan gagal karena belum memenuhi syarat nominal minimal.

Sebaliknya, jika nominal sudah mencapai Rp10.000 atau lebih, sistem melanjutkan ke pemeriksaan kedua untuk membandingkan nominal tersebut dengan sisa saldo yang dimiliki pengguna. Apabila nominal yang ditarik melebihi saldo yang tersedia, transaksi kembali dibatalkan sambil memunculkan pesan bahwa saldo tidak mencukupi beserta informasi jumlah saldo saat itu.

Jika nominal memenuhi syarat batas minimum dan saldo mencukupi, sistem langsung memproses penarikan dengan memotong nilai saldo sebesar nominal yang ditarik.

Terakhir, program menampilkan konfirmasi bahwa transaksi tarik tunai telah berhasil dilakukan, lengkap dengan nominal yang ditarik serta sisa saldo terbaru pengguna.

2. Tantangan Multi-akun

```

1 package Pekan1;
2 import java.util.ArrayList;
3 import java.util.Scanner;
4 import java.text.DecimalFormat;
5
6 public class Main {
7     public static void main(String[] args) {
8         Scanner input = new Scanner(System.in);
9         ArrayList<Rekening> daftarRekening = new ArrayList<>(); // Collection untuk menyimpan semua rekening
10        Rekening akunAktif = null; // Objek belum diinisialisasi (null)
11        boolean isRunning = true;
12
13        System.out.println("=== SISTEM PERBANKAN MINI ===");
14
15        while (isRunning) {
16            System.out.println("\nMenu Utama : ");
17            System.out.println("1. Buka Rekening Baru");
18            System.out.println("2. Setor Tunai");
19            System.out.println("3. Tarik Tunai");
20            System.out.println("4. Cek Informasi Rekening");
21            System.out.println("5. Ganti Akun");
22            System.out.println("0. Keluar");
23            System.out.print("Pilih Menu : ");
24
25            int pilihan = input.nextInt();
26            input.nextLine();
27
28            switch (pilihan) {
29                case 1 :
30                    System.out.print("Masukkan No Rekening : ");
31                    String no = input.nextLine();
32                    System.out.print("Masukkan Nama Pemilik : ");
33                    String nama = input.nextLine();
34                    System.out.print("Masukkan Saldo Awal : ");
35                    double saldo = input.nextDouble();
36                    input.nextLine();
37
38                    // Instansiasi Object / Menjalankan Constructor
39                    Rekening rekeningBaru = new Rekening(no, nama, saldo);
40                    daftarRekening.add(rekeningBaru); // Tambahkan ke ArrayList, bukan menimpa
41                    akunAktif = rekeningBaru; // Rekening baru otomatis jadi akun aktif
42                    break;
43
44                case 2 :
45                    if (akunAktif == null) {
46                        System.out.println("Error: Mohon maaf, anda belum memiliki nomor rekening!");
47                    } else {
48                        System.out.print("Masukkan nominal setor: ");
49                        double setor = input.nextDouble();
50                        akunAktif.setorTunai(setor); // Memanggil Behavior / method
51                    }
52                    break;
53
54                case 3 :
55                    if (akunAktif == null) {
56                        System.out.println("Error: Mohon maaf, anda belum memiliki nomor rekening!");

```

```

57         } else {
58             System.out.print("Masukkan nominal tarik: ");
59             double tarik = input.nextDouble();
60             akunAktif.tarikTunai(tarik);
61         }
62         break;
63
64     case 4 :
65         if (akunAktif == null) {
66             System.out.println("Error: Anda belum membuka rekening!");
67         } else {
68             akunAktif.cekInformasi();
69         }
70         break;
71
72     case 5 : // Bonus: Ganti Akun
73         if (daftarRekening.isEmpty()) {
74             System.out.println("Error: Belum ada rekening yang terdaftar!");
75             break;
76         }
77         System.out.print("Masukkan nomor rekening yang ingin diaktifkan: ");
78         String noDicari = input.nextLine();
79
80         Rekening hasilCari = null;
81         for (Rekening r : daftarRekening) {
82             if (r.nomorRekening.equals(noDicari)) {
83                 hasilCari = r;
84                 break;
85             }
86         }
87
88         if (hasilCari != null) {
89             akunAktif = hasilCari;
90             System.out.println("Berhasil! Akun aktif sekarang: " + akunAktif.nomorRekening + " (" + akunAktif.namaPemilik + ")");
91         } else {
92             System.out.println("Error: Nomor rekening tidak ditemukan!");
93         }
94         break;
95
96     case 0 :
97         isRunning = false;
98         System.out.println("Sistem ditutup. Terima Kasih!");
99         break;
100
101     default:
102         System.out.println("Pilihan tidak valid!");
103     }
104 }
105 input.close();
106 }
107 }

```

Perubahan yang dilakukan pada kelas Main.java bertujuan untuk mengatasi permasalahan penyimpanan data rekening yang sebelumnya hanya mampu menampung satu objek dalam satu waktu. Pada versi awal, program hanya menggunakan sebuah variabel tunggal bertipe Rekening bernama akunAktif untuk menyimpan data rekening yang sedang aktif. Akibatnya, setiap kali pengguna memilih menu "Buka Rekening Baru", objek Rekening yang lama akan langsung digantikan oleh objek yang baru, sehingga data rekening sebelumnya hilang dan tidak dapat diakses kembali.

Untuk mengatasi hal tersebut, ditambahkan sebuah struktur data koleksi berupa `ArrayList<Rekening>` yang diberi nama `daftarRekening`. Struktur data ini berfungsi sebagai wadah penyimpanan yang mampu menampung banyak objek Rekening sekaligus dan dapat bertambah secara dinamis seiring bertambahnya jumlah rekening yang dibuat

oleh pengguna. Dengan adanya daftarRekening, setiap objek rekening yang dibuat tidak lagi saling menimpa, melainkan tersimpan secara permanen selama program berjalan. Variabel akunAktif sendiri tetap dipertahankan dan tidak dihapus, karena perannya berbeda dengan daftarRekening. Jika daftarRekening berfungsi sebagai basis data yang menyimpan seluruh rekening yang pernah dibuat, maka akunAktif berperan sebagai penunjuk (referensi) terhadap satu rekening tertentu yang sedang digunakan untuk melakukan transaksi seperti setor tunai, tarik tunai, maupun pengecekan informasi.

Perubahan logika kemudian dilakukan pada bagian menu "Buka Rekening Baru". Objek Rekening yang baru dibuat terlebih dahulu ditampung ke dalam variabel sementara bernama rekeningBaru, kemudian objek tersebut ditambahkan ke dalam daftarRekening menggunakan method add(). Setelah itu, barulah objek yang sama dijadikan sebagai akunAktif, sehingga pengguna dapat langsung melakukan transaksi terhadap rekening yang baru saja dibuat tanpa kehilangan data rekening-rekening sebelumnya yang sudah tersimpan di dalam ArrayList.

Sebagai fitur tambahan (bonus), ditambahkan pula menu baru bernomor lima yang diberi nama "Ganti Akun". Menu ini memungkinkan pengguna untuk berpindah dari satu rekening aktif ke rekening lain yang telah terdaftar sebelumnya, dengan cara memasukkan nomor rekening yang ingin diaktifkan. Program kemudian melakukan proses pencarian dengan menelusuri seluruh isi daftarRekening menggunakan perulangan for-each, lalu membandingkan nomor rekening setiap objek dengan nomor yang dimasukkan pengguna menggunakan method equals(), mengingat perbandingan data bertipe String sebaiknya dilakukan berdasarkan isi datanya dan bukan berdasarkan referensi objeknya. Apabila objek dengan nomor rekening yang sesuai ditemukan, maka objek tersebut akan disimpan sebagai akunAktif yang baru. Sebagai langkah antisipasi, ditambahkan pula pengecekan kondisi apabila daftarRekening masih kosong, sehingga proses pencarian tidak akan dijalankan apabila belum ada satu pun rekening yang pernah dibuat oleh pengguna.

Adapun bagian menu lainnya, yaitu menu "Setor Tunai", "Tarik Tunai", "Cek Informasi Rekening", dan "Keluar", tidak mengalami perubahan logika sama sekali. Hal ini dikarenakan keempat menu tersebut memang dirancang untuk beroperasi terhadap satu objek akunAktif yang sedang aktif, bukan terhadap keseluruhan data yang tersimpan di dalam daftarRekening.